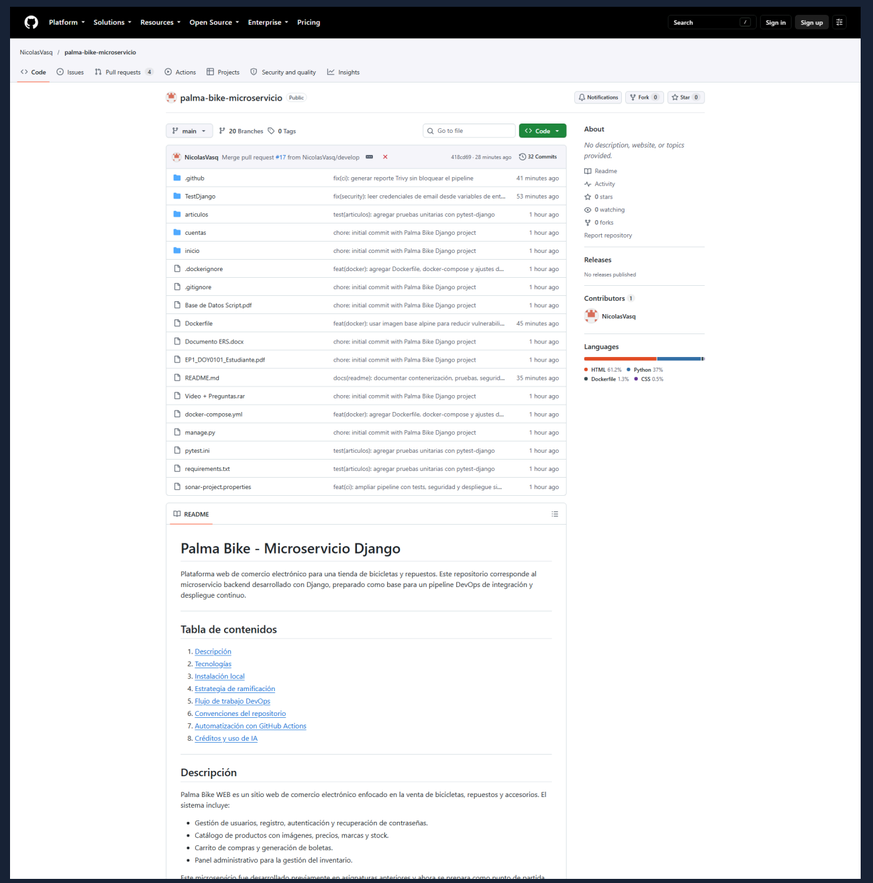
Evaluación Parcial N°2

Pipeline CI/CD para el microservicio Palma Bike
Ingeniería DevOps (DOY0101)

Nicolas Vasquez

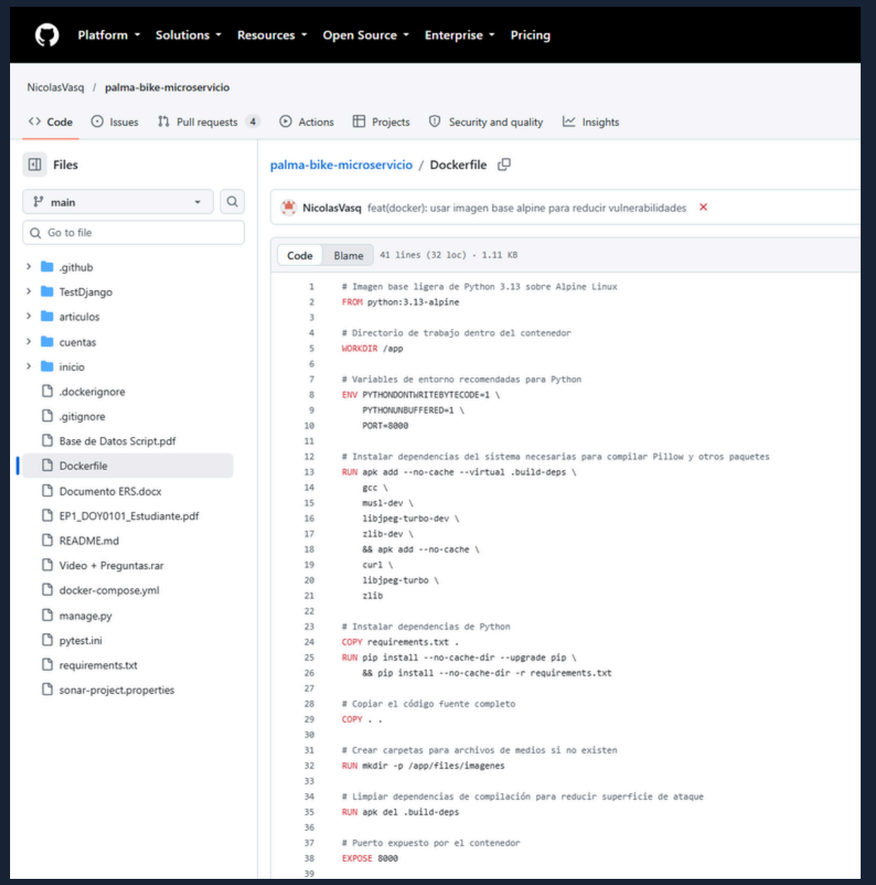
Objetivos de la evaluación

- Contenerizar el microservicio Django con Docker.
- Implementar un pipeline CI/CD en GitHub Actions.
- Integrar pruebas automatizadas con pytest-django.
- Incorporar análisis de seguridad: Bandit, Trivy y Dependabot.
- Automatizar el despliegue en un entorno cloud simulado.
- Orquestar los contenedores con Docker Compose.



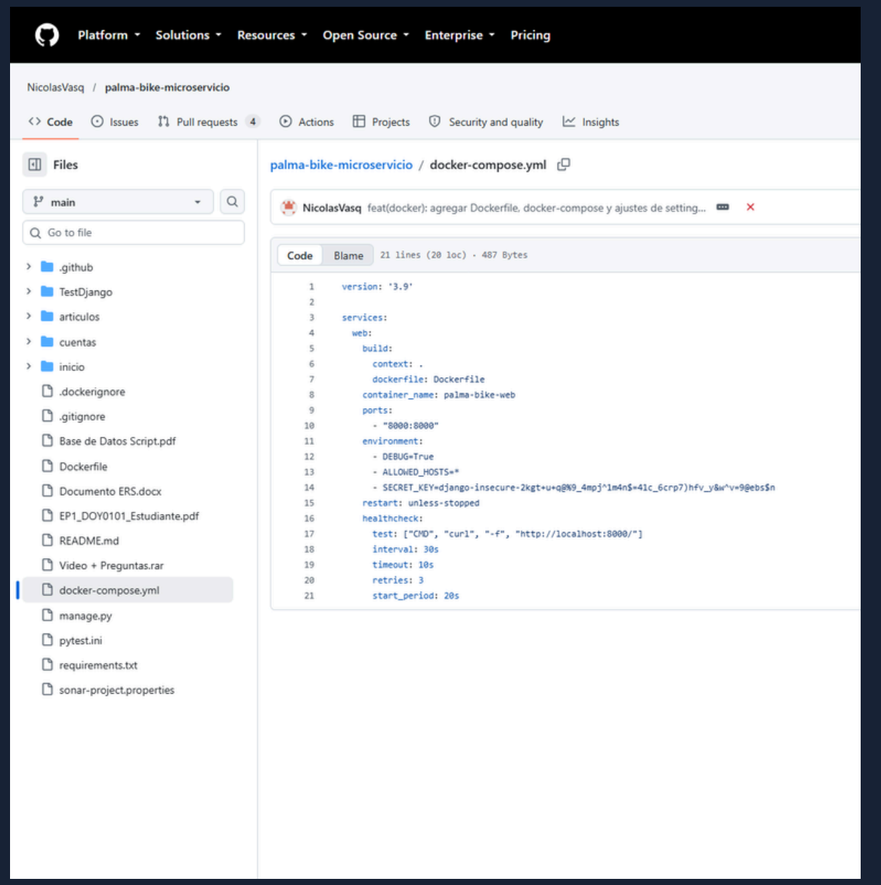
Contenerización con Docker

- Dockerfile basado en python:3.13-alpine.
- Imagen ligera, con dependencias de sistema mínimas.
- Se instalan Pillow y sus librerías nativas.
- Se eliminan paquetes de compilación para reducir la superficie de ataque.
- .dockerignore evita subir archivos innecesarios a la imagen.



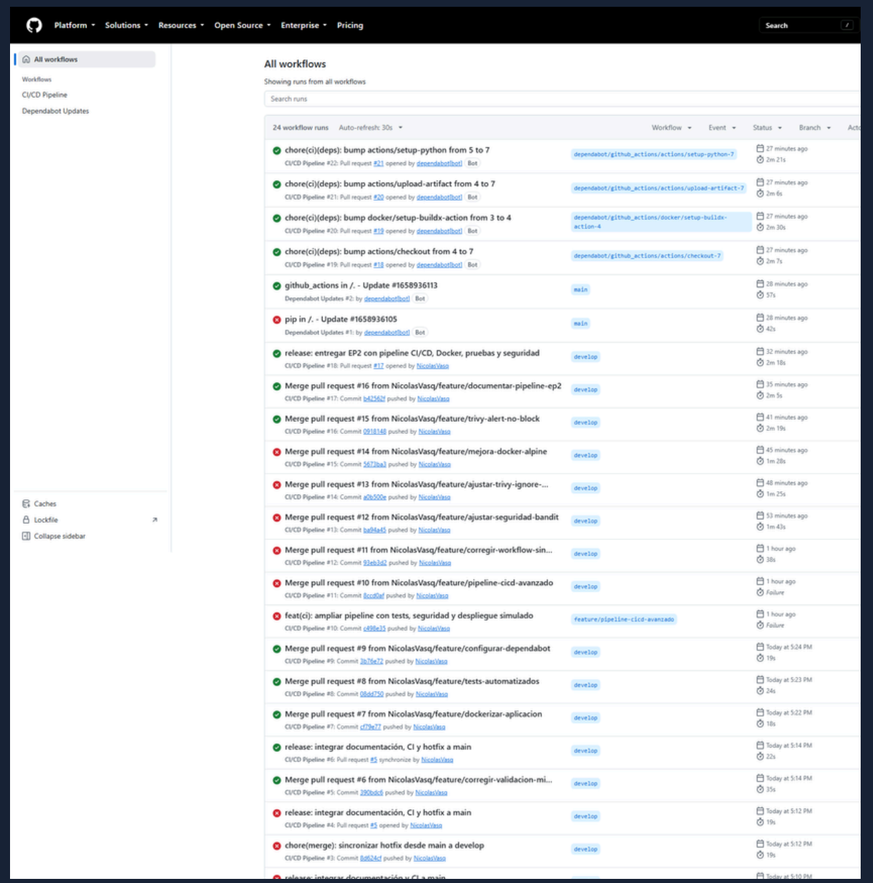
Orquestación con Docker Compose

- docker-compose.yml levanta el servicio web.
- Define variables de entorno sensibles.
- Incluye healthcheck con curl.
- Política de reinicio unless-stopped.
- Permite escalar horizontalmente si se agregan servicios.



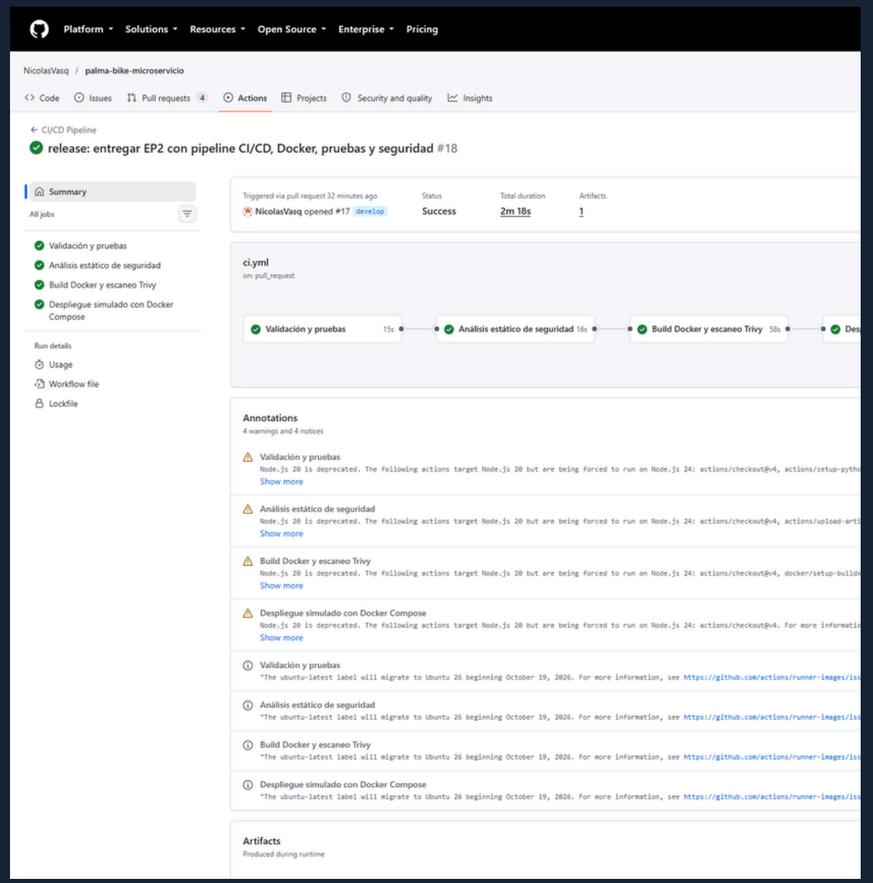
Pipeline CI/CD en GitHub Actions

- Disparadores: push a develop y pull request a main.
- Jobs: validación/pruebas, seguridad de código, build + scan, despliegue simulado.
- Cada job depende del anterior: si falla un paso, se bloquea el pipeline.
- Trazabilidad completa desde el commit hasta el despliegue simulado.



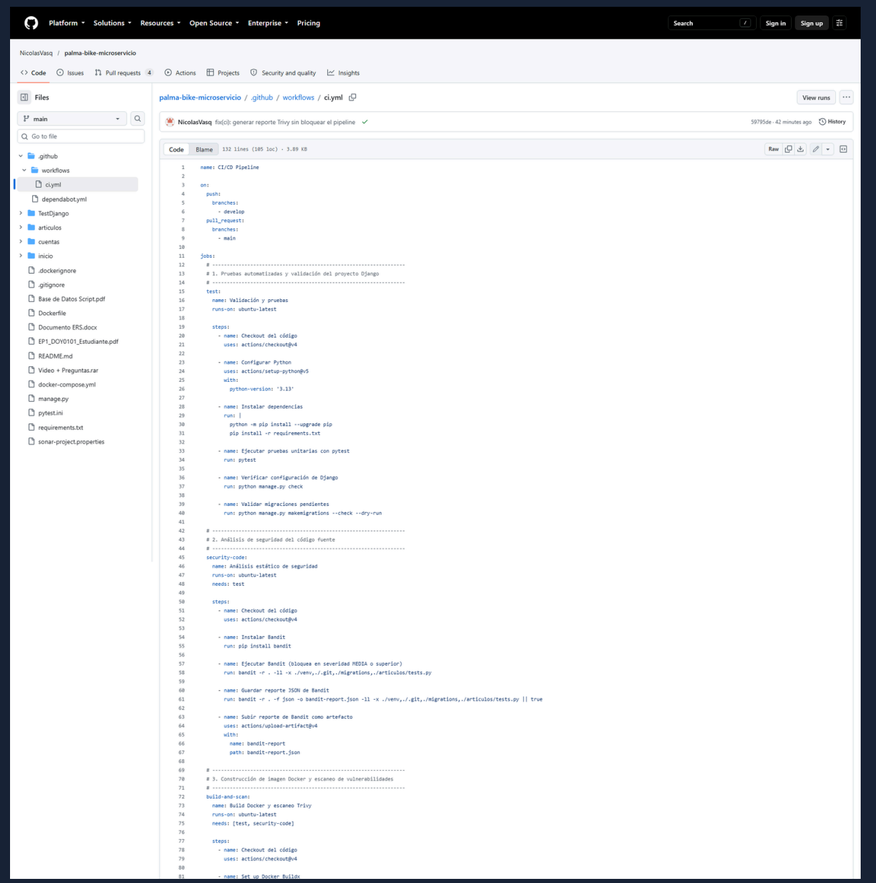
Pruebas automatizadas

- Framework: pytest-django.
- Pruebas de modelos: Marca, Producto, stock no negativo.
- Pruebas de vistas: inicio y nosotros responden HTTP 200.
- Se ejecutan en cada pipeline antes de continuar.



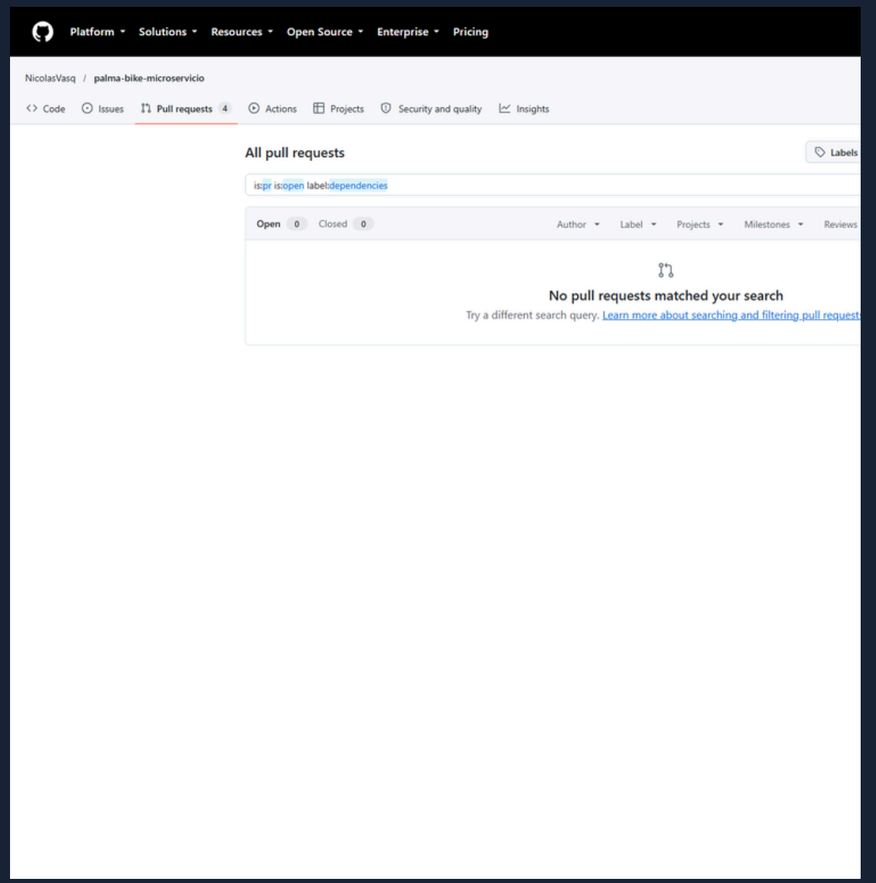
Seguridad en el pipeline

- Bandit: análisis estático de código Python.
- Trivy: escaneo de vulnerabilidades en imagen Docker.
- Dependabot: genera PRs automáticos de actualización.
- Bloqueo automático si Bandit detecta problemas de severidad media o superior.



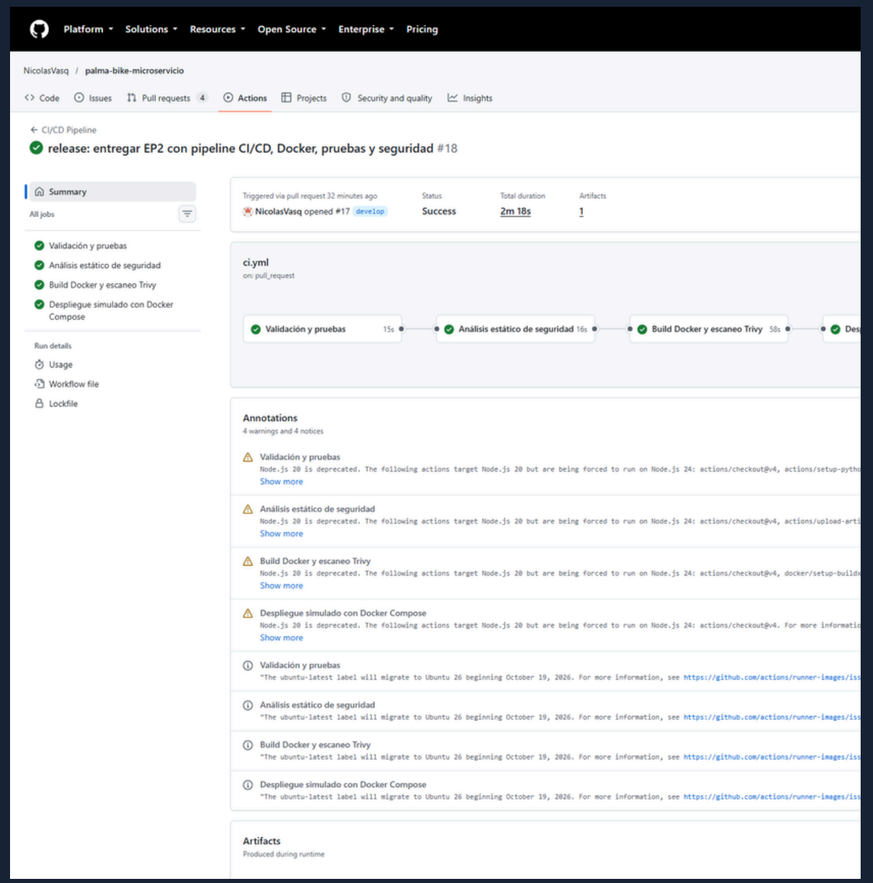
Gestión de dependencias con Dependabot

- Configurado vía .github/dependabot.yml.
- Escanea requirements.txt y GitHub Actions semanalmente.
- Abre automáticamente pull requests con actualizaciones.
- Evidencia: PRs #18 a #21 ya creados por Dependabot.



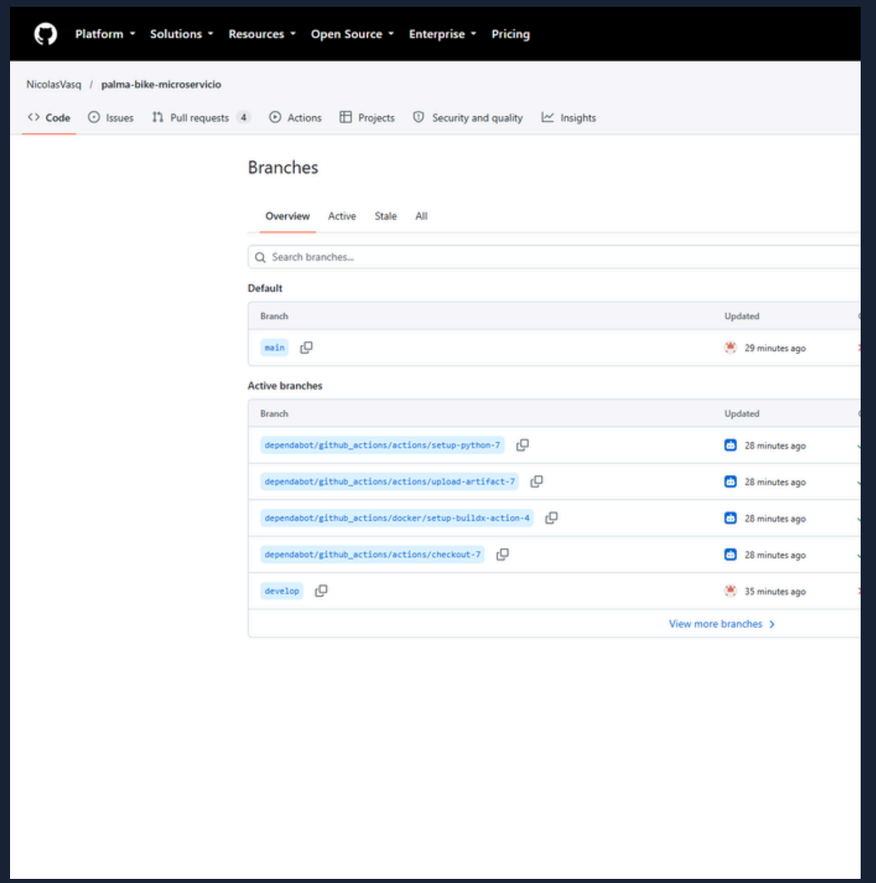
Despliegue automatizado simulado

- Último job del pipeline.
- Levanta el servicio con docker compose up -d --build.
- Verifica salud con curl a http://localhost:8000/.
- Ejecuta python manage.py check dentro del contenedor.
- Detiene los servicios al finalizar.



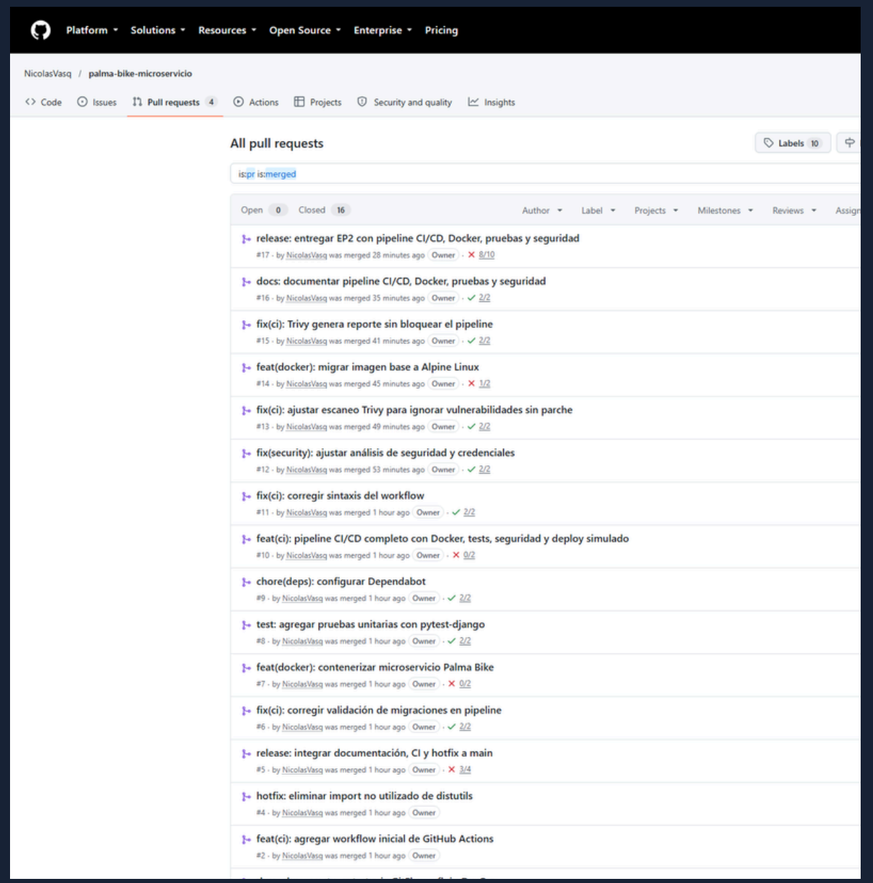
Estrategia de ramificación GitFlow

- Ramas principales: main y develop.
- Ramas de soporte: feature/* y hotfix/*.
- Cambios integrados mediante pull requests.
- Más de 10 PRs mergeados durante la evaluación.



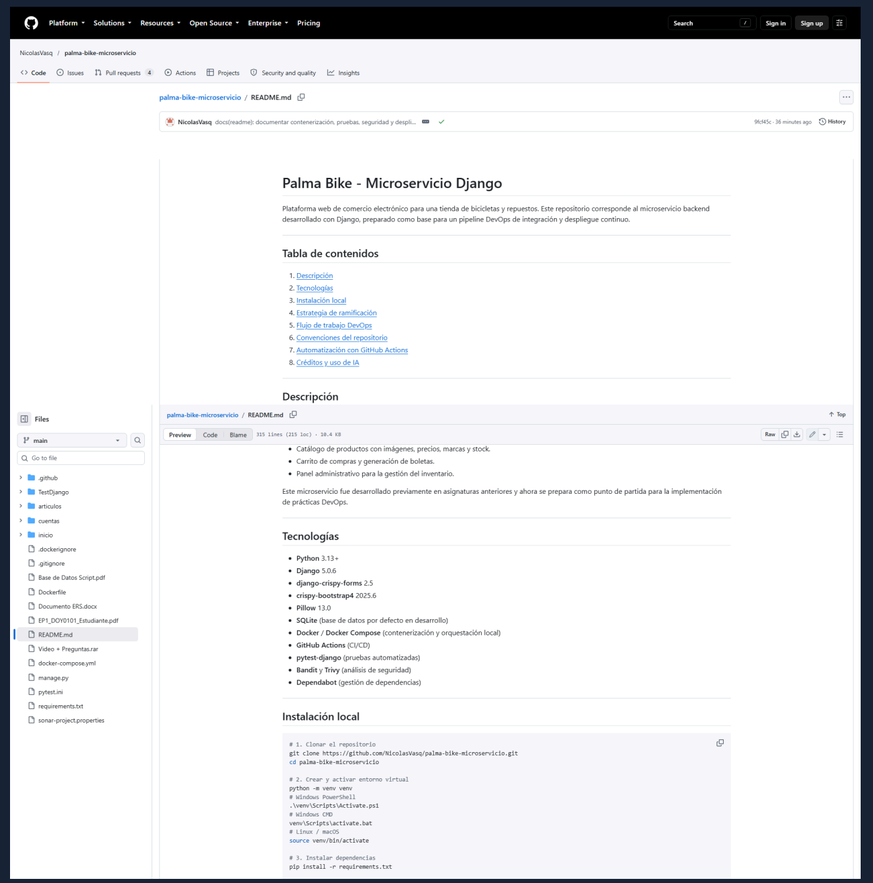
Pull Requests y trazabilidad

- Cada cambio documentado con título y descripción.
- Convención de commits Conventional Commits.
- Merge mediante pull requests a develop y luego a main.
- Historia clara y rastreable en GitHub.



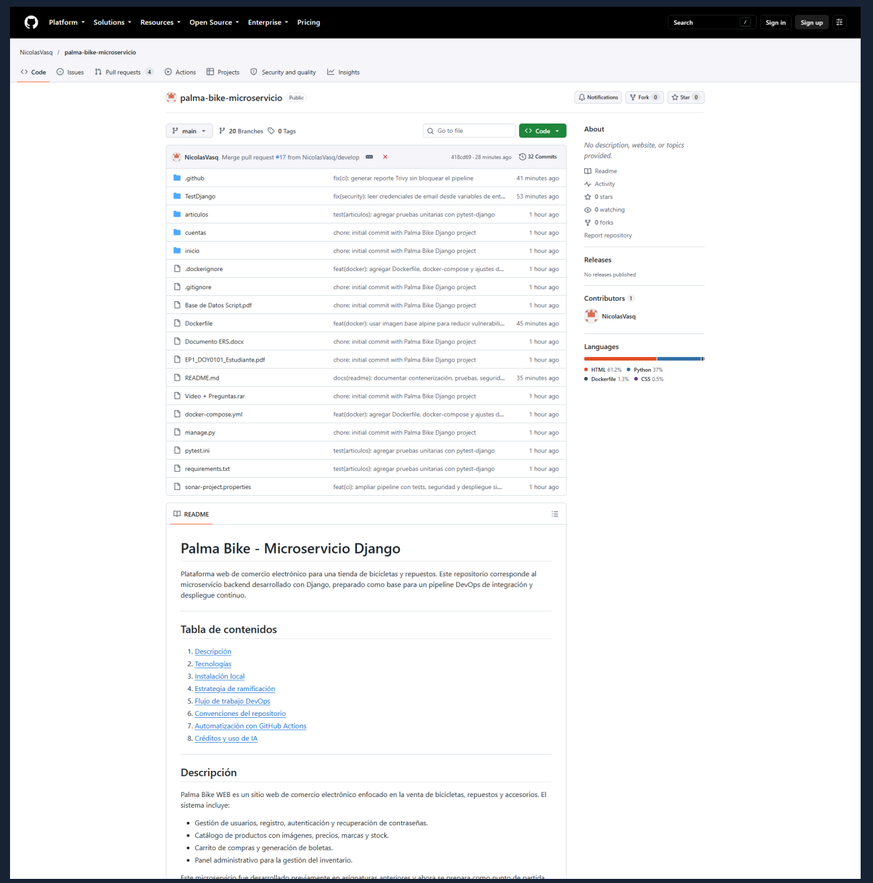
Documentación del pipeline

- README.md incluye: instalación, Docker, Docker Compose.
- Sección de pruebas automatizadas y seguridad.
- Variables de entorno requeridas.
- Explicación del despliegue simulado.



Resultados

- Pipeline CI/CD funcional y aprobado en main.
- Imagen Docker construida y escaneada.
- Pruebas automatizadas pasando.
- Despliegue simulado exitoso en cada ejecución.
- Dependabot activo generando actualizaciones.



Conclusiones

- Se construyó un pipeline CI/CD completo sobre el microservicio.
- Se integraron contenedores, pruebas, seguridad y despliegue simulado.
- El flujo GitFlow permite trabajo colaborativo y trazable.
- El repositorio está listo para escalar hacia un despliegue real.

Muchas gracias

¿Preguntas?